

PetNTester AI Audit Remediation Guide

The PetNTester AI tool must evolve into a true multi-tool security framework with automated end-to-end exploit chaining and comprehensive coverage. Drawing on architectures like HexStrike-AI and modern security orchestration platforms, the remediation plan focuses on a **central orchestrator** that integrates scanners (Nmap, Nuclei, SQLMap, etc.) as callable services, triggers chained exploits (e.g. SQLi → SQLMap dump), and minimizes noise through context-aware filtering ¹ ². The design should treat each scanner as a microservice: wrapped in an interface (e.g. an MCP or RPC endpoint) so the orchestration “brain” can invoke it programmatically ³ ⁴. The result is a scalable, concurrent pipeline where tools run in parallel containers or jobs, with results automatically parsed and fed into the next stage of the workflow ⁴ ⁵.

Tool Orchestration Architecture

The core engine must **orchestrate multiple scanners** under a unified workflow. Each tool (Nmap, Nuclei, SQLMap, etc.) is packaged as a service or module with a defined API. For example, frameworks like HexStrike-AI use a FastMCP server as a communication hub, exposing each tool as a decorated function that AI agents invoke ³. Embedding scanners in containers (or via Docker Compose) and using an orchestration pipeline (Kubernetes jobs, Airflow DAGs, etc.) allows simultaneous execution and reuse of results. Figure 1 illustrates this pattern: an Nmap module function is exposed for orchestration. The orchestrator can then call `nmap_scan(target, ports)` directly, removing manual parsing and ensuring consistency.

```
@mcp.tool()
def nmap_scan(target: str,
             scan_type: str = "-sV",
             ports: str = "",
             additional_args: str = "") -> Dict[str, Any]:
    result = hexstrike_client.safe_post("api/tools/nmap", data)
```

Figure 1: Example of exposing an Nmap scan function as an MCP tool endpoint (HexStrike-AI style) ³. Each scanner is wrapped so the orchestrator can invoke it programmatically.

This orchestration layer should manage tool dependencies. For instance, after port enumeration with Nmap, the orchestrator can automatically trigger Nuclei templates against discovered hosts and ports. Scans should be scheduled with retry logic and concurrency controls to balance throughput and resource usage ⁶ ². Outputs from each tool must be normalized (e.g., as JSON records) and fed into a central aggregator, similar to how modern STO (Security Testing Orchestration) frameworks **aggregate and deduplicate** findings from all scanners ² ⁷. This ensures that the pipeline can handle hundreds of scans in parallel while maintaining a unified view of results.

Chained Exploitation Workflow

The system must **automate vulnerability chains** rather than treating each finding in isolation. In practice, when a low-impact finding (e.g. error message or parameter clue) is detected, the engine should ask “what can this be combined with?” as real attackers do ⁸ ⁹. For example, upon detecting

a SQL injection issue (via a Nuclei or custom probe), the orchestrator would automatically launch SQLMap with the relevant target URL/parameter to extract data. This chaining is made reliable through resilient code: implement retry mechanisms, timeouts, and failovers so that a missed exploit attempt can be retried or a fallback is triggered ⁶. The orchestrator effectively “translates intent to execution”: a high-level command (e.g. “dump database on victim.com”) is broken down into sequenced tool calls ¹⁰.

Crucially, the chain must record context at each step. A successful exploit (like a database dump) feeds back into the central context store, enabling further stages (e.g. crawling extracted URLs or pivoting on credentials). Automated post-exploit checks (e.g. verify login or RCE achieved) should confirm success. The chain logic should also include **false-positive suppression**: for instance, skip triggering SQLMap on inputs that pass static filters, and ignore theoretical flows if no real vulnerability can be demonstrated ¹¹. In other words, each link in the chain must be validated before proceeding to the next, to avoid wasted effort on non-exploitable findings.

Inter-Module Communication & Scan Context

All modules must **communicate via a shared bus or context object**, enabling loose coupling and scalability. A common pattern is a message queue (e.g. Redis Pub/Sub, RabbitMQ, Kafka) where modules publish events (discovered hosts, found vulnerabilities, exploit results) and subscribe to topics of interest. For example, one architecture used by an AI pentesting system has agents coordinating through a Redis pub/sub channel ¹². This lets the Nmap module publish “open_port” events that trigger the Nuclei or SQLMap modules. Alternatively, use a persistent **ScanContext** object (in memory or a database) that accumulates all scan metadata for a given target. This context tracks enumerated hosts, open ports, session cookies, credentials, etc., so each module has access to the current state of the assessment.

Regardless of the mechanism, inter-module messaging should be asynchronous and event-driven. The orchestration core (acting as a broker) routes messages based on content; for example, a Nuclei finding of `SQL_INJECTION` can automatically enqueue a SQLMap job on that URL. The implementation can follow the HexStrike-AI model, where a “FastMCP” server binds agent code to tool functions ³, or simply a custom event-handler that triggers child tasks. The key is **loose coupling** – modules publish well-defined outputs (e.g. JSON with standardized fields) and listen for relevant inputs. This design also aids debugging and scaling: engineers can add new modules (e.g. XSS fuzzer) without rewiring the entire system.

Codebase Cleanup, Bug Fixes, and Refactoring

The remediation must aggressively **clean up and refactor** the code. All known critical bugs (especially logic errors in scan coordination or exploit launching) should be fixed first. Static analysis and testing should target concurrency issues, input validation errors, and any security flaws in the tool wrappers. In parallel, identify and remove **dead or unreachable code**: unused functions, outdated scan paths, or commented-out logic. Dead code “occupies valuable space and can hinder performance, maintainability, and even security” ¹³. Removing it will reduce complexity and potential maintenance errors.

Next, refactor to **eliminate duplication** and centralize common helpers. For instance, if multiple modules construct HTTP requests or parse responses, factor that into a shared utilities module. Centralize configuration loading (e.g. target list, proxy settings) and result formatting. Adopt consistent error-handling patterns across modules. Wherever similar code exists (e.g. JSON parsing, logging),

extract it into a common library. This not only reduces code bloat but also ensures that improvements (like enhanced logging or retry logic) are applied uniformly. In short, aim for a clean architecture (e.g. apply SOLID principles) so that each component has a single responsibility and shares robust core utilities.

Incremental Roadmap & Testing Strategy

Implement these changes **incrementally** with frequent testing. A suggested sequence is:

1. **Foundation:** Build or harden the orchestrator core (CI/CD jobs or controller). Integrate Redis/Kafka for messaging. Write unit tests for message parsing and simple tool invocation.
2. **Tool Wrappers:** Add one tool at a time (start with Nmap, then Nuclei, then SQLMap). For each, implement the MCP/service wrapper, ensure its output is captured and normalized. Write integration tests (e.g. mock a target and verify correct commands are run).
3. **Chaining Logic:** Enable basic chaining: e.g. after Nmap discovery, automatically launch a port-specific scan. Once proven, add logic for vulnerability-to-exploit triggers (e.g. trigger SQLMap on detected SQLi). Test these flows end-to-end in an isolated lab.
4. **Context & Communication:** Develop the ScanContext object or database. Update modules to read/write context (e.g. discovered endpoints, credentials). Validate that context is correctly shared.
5. **Coverage Fuzzers:** Incorporate endpoint and parameter fuzzing (see below). Ensure fuzzers consume targets from context and publish new findings back into context.
6. **Stabilization:** Fix any bugs found, tune timeouts/rate limits, and ensure the system can run unattended on a test network. Add monitoring to detect deadlocks or runaway scans.

At each step, include automated tests or dummy targets. Use containerized target services (e.g. vulnerable web apps) to simulate exploit chains safely. This phased approach minimizes risk: core services and communications are validated before adding complex exploits.

Endpoint & Parameter Coverage Best Practices

To achieve **complete coverage of discovered endpoints and parameters**, the workflow should incorporate automated fuzzing and discovery at each stage. After initial host and port enumeration, use tools like **ffuf** for directory fuzzing on each live endpoint ¹⁴. The Medium Recon Playbook notes: *“With live endpoints in hand, it’s time to fuzz directories and parameters to find hidden resources”* ¹⁴. In practice, this means feeding each active URL into ffuf with common path wordlists (skipping typical 404 responses) to discover unlinked files or API endpoints.

Similarly, perform parameter fuzzing: once an endpoint is identified, use ffuf or specialized tools (e.g. Arjun or Burp’s Param Miner) to try common parameter names and values ¹⁵ ¹⁶. As Dana Epp advises, *“By leveraging fuzzing and tools such as Param Miner, [testers] can systematically uncover undocumented and hidden parameters”* ¹⁶. Incorporate header and JSON field fuzzing as needed (e.g. ffuf’s `-H` and `-d` flags). All newly discovered endpoints or parameters should be added to the ScanContext and re-scanned with relevant templates or payloads.

Finally, correlate results to reduce noise. Normalize all findings (e.g. each vulnerability record should include the exact URL, parameter, payload, and response info). Deduplicate similar findings by asset and signature ¹⁷. Contextual filters should suppress likely false positives: for example, ignore scanners reporting issues on endpoints already proven safe by authenticated scans or covered by compensating

controls ¹¹. This ensures the tool spends minimal effort on dead-ends and focuses on actionable chains.

Prioritized Remediation Tasks

Priority	Task / Feature	Notes / References
Critical	Orchestrator & Multi-tool Integration: Implement central orchestration engine with service wrappers for Nmap, Nuclei, SQLMap, etc. Ensure containerized concurrency and CI/CD integration ⁴ ³ .	See [HexStrike-AI architecture][22†L313-L318], [Harness STO][11†L132-L136]
Critical	Exploit Chain Automation: Automate end-to-end exploit flows (e.g. SQLi → SQLMap dump). Add retry logic and state tracking for each step ⁶ ⁹ . Validate each link before proceeding.	Use [CheckPoint analysis] [27†L335-L338], chain correlation [25†L268-L273]
High	Module Communication (Pub/Sub): Build inter-module messaging (Redis/Kafka) or shared ScanContext. Modules should publish findings and trigger subscribers (e.g. SQL findings triggering exploit module) ¹² ³ .	Adopt event-driven design (see [Sploitus example] [7†L447-L450])
High	Bug Fixes & Cleanup: Address all critical bugs in scanning/exploit logic. Remove unreachable/dead code to improve maintainability ¹³ . Add logging and exception handling uniformly.	Dead code removal as recommended by [vFunction][39†L129-L131]
High	Output Normalization & Deduplication: Normalize outputs (JSON) and dedupe findings by asset and signature ¹⁷ . Correlate multi-stage results to suppress false positives ¹¹ .	Use built-in dedupe (STO) or custom logic ⁷ ¹¹
Medium	Refactoring & Helpers: Refactor common logic into shared helper modules (HTTP clients, parsers, templates). Eliminate code duplication for maintainability.	Follow clean-code principles; use static analysis tools.
Medium	Endpoint/Param Fuzzing: Integrate directory/parameter fuzzing on discovered endpoints (ffuf, Param Miner) ¹⁴ ¹⁶ . Ensure all findings feed back into ScanContext.	See [Recon Playbook] [48†L218-L225] and Dana Epp [Param Miner][51†L229-L236] guides.
Low	Incremental Testing Pipeline: Develop CI jobs or test harnesses. Verify each component in isolation before full integration.	Shift-left scanning approach ² .

Component/Module Instructions

- **Orchestrator (core engine):** Build a scheduler that pulls tasks (scan jobs) from a queue. Use an async worker pool (e.g. asyncio, Celery). Integrate a pub/sub broker (Redis/Kafka) for inter-module messages. Implement logic to interpret high-level commands (like “scan all domains”) into sequenced tool calls. Include global retry and timeout logic.

- **Nmap Module:** Wrap Nmap calls in a dedicated function (as in Figure 1). Ensure it can take IP ranges or host lists from ScanContext. Parse output (XML/JSON) into structured records and publish “host.up” and “port.open” events. Add error handling for permission/privilege issues (running with `--privileged`).
- **Nuclei Module:** Configure Nuclei templates and integrate via a subprocess or API. After Nmap publishes open HTTP ports, enqueue Nuclei scans on those targets. Normalize each finding (CVE/description, severity). Publish findings to the central context.
- **SQLMap Module:** Trigger this module automatically when a SQLi signature or anomaly is detected by Nuclei or custom probes. Pass the target URL/parameter and required options. After SQLMap completes, capture the dumped data path or output. Store results and mark the vulnerability as verified.
- **ScanContext / Data Store:** Define a data model (in-memory or DB) for `ScanContext` that holds targets, findings, credentials, and state flags. Update context after each module: e.g. `context.add_endpoint(url)`, `context.add_vuln(vuln)`. Ensure thread-safe access if concurrent. Persist context for long-running scans so progress survives restarts.
- **Pub/Sub Integration:** Write small handlers/subscribers for each event type (e.g. `on_host_discovered`, `on_vuln_found`). Ensure that when one handler posts an event, interested modules receive it. For example, subscribe the SQLMap module to “SQLI_FOUND” events. Use JSON schemas to validate message formats.
- **Utilities/Helpers:** Centralize common functions here. HTTP request functions, JSON parsing, logging setup, config management (CLI args, env vars). Include a unified error-reporting mechanism. Remove any duplicate code in these utilities and ensure they are used by all modules.
- **Fuzzing Support:** Add modules or integrate libraries for discovery: e.g., call ffuf or Param Miner internally. Automate loading of wordlists (extend with custom lists over time). After fuzzing completes, feed new URLs/parameters back into the pipeline (e.g. as hosts for Nuclei scans).

By following this guide, the engineering team can incrementally transform PetNTester into a robust, scalable framework. Each section above is supported by industry best practices (e.g. containerized multi-tool orchestration ⁴ ³, event-driven workflows ¹², and systematic fuzzing for full coverage ¹⁴ ¹⁶), ensuring the future codebase is efficient, maintainable, and effective at finding and exploiting chained vulnerabilities.

Sources: Authoritative articles and documentation on security orchestration, exploit automation, and best practices have been cited throughout (e.g. ³ ⁹ ¹⁴) to ground these recommendations in proven designs. Each actionable item above can be traced back to these industry references for further detail.

¹ ³ ⁶ ¹⁰ Hexstrike-AI: LLM Orchestration Driving Real-World Zero-Day Exploits
<https://blog.checkpoint.com/executive-insights/hexstrike-ai-when-llms-meet-zero-day-exploitation/>

² ⁷ Harness Security Testing Orchestration (STO) Overview | Harness Developer Hub
<https://developer.harness.io/docs/security-testing-orchestration/overview/>

- 4 GitHub - FuzzingLabs/mcp-security-hub: A growing collection of MCP servers bringing offensive security tools to AI assistants. Nmap, Ghidra, Nuclei, SQLMap, Hashcat and more.
<https://github.com/FuzzingLabs/mcp-security-hub>
- 5 GitHub - ElonMusk2002/Automated-Web-Scanner-with-AI-Integration: AI in Pentest: Automated Web Scanner with AI Integration
<https://github.com/ElonMusk2002/Automated-Web-Scanner-with-AI-Integration>
- 8 Chain Exploitation in Web Pentesting | by Esra Kayhan | Jan, 2026 | Medium
<https://medium.com/@esrakyhn.u/chain-exploitation-in-web-pentesting-%EF%B8%8F-ef776dc44a87>
- 9 11 How to Automate Vulnerability Management Steps, Tools, and Benefits - Palo Alto Networks
<https://www.paloaltonetworks.com/cyberpedia/automating-vulnerability-management>
- 12 Exploit for CVE-2023-1234
<https://sploit.us.com/exploit?id=94E07749-2E3B-5055-A6CE-12ABD0B09A88>
- 13 Exposing dead code: strategies for detection and elimination
<https://vfunction.com/blog/dead-code/>
- 14 15 The Recon Playbook: Finding Hidden Endpoints Like a Pro | by Maxwell Cross | Medium
<https://medium.com/@maxwellcross/the-recon-playbook-finding-hidden-endpoints-like-a-pro-a48b8dea3d3f>
- 16 Finding hidden API parameters
<https://danaepp.com/finding-hidden-api-parameters>
- 17 The Complete Guide to Vulnerability Scanning (2025 Edition)
<https://www.vicarius.io/articles/the-complete-guide-to-vulnerability-scanning-2025-edition>